Build • Ship • Run • Scale

One Notebook...
Everything about Docker!

The Docker Complete Handbook
Basic to Advanced



✓ Learn Docker Step by Step!

- Docker Basics
- Docker CLI
- Dockerfile
- Images & Containers
- Networking
- Volumes & Storage
- Docker Compose
- Security Best Practices
- Registries (Docker Hub, ECR)
- CI/CD + Production Setup
- Troubleshooting
- Interview Questions

✓ Simple
✓ Practical
✓ Real-World
✓ Production Ready

From Zero to Pro



- Commands
- Examples
- Best Practices
- Real Use Cases
- Interview Q&A

Containers make DevOps Faster, Simpler and Better! ♥

Let's Master Docker Together! 🔥

@the_devops_journey_with_sanket

Date : 08 / 08 / 2025 @the_devops_journey_with_sanket

What is Docker ?

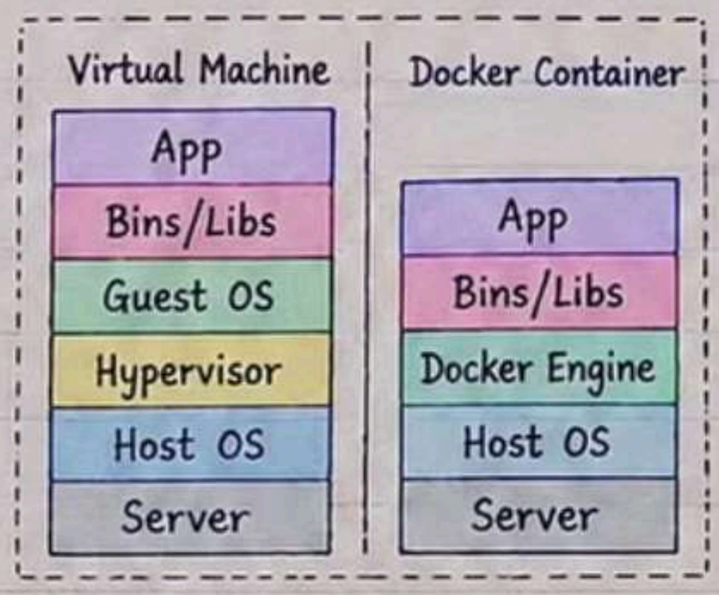


→ **Simple Definition :**

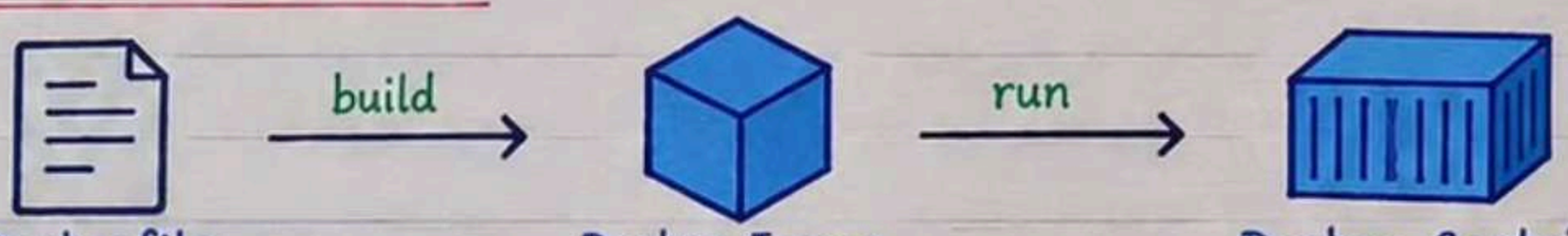
Docker is an open-source platform that helps us build, ship and run applications inside lightweight, portable containers.
It packages the application + its dependencies + runtime into a single unit called a container.

→ **Why Docker ? (Key Benefits)**

- ✔ Consistency : Works the same on every machine (Dev/QA/Prod)
- ✔ Portability : Build once, run anywhere
- ✔ Isolation : Each app runs in its own container
- ✔ Lightweight : Uses less resources than a VM
- ✔ Faster Deployment : Start containers in seconds



→ **How Docker Works ?**



Dockerfile (Instructions) → Docker Image (Read-only template) → Docker Container (Running instance)

→ **Common Docker Commands :**

```
docker --version                  # Check Docker version
docker build -t myapp .           # Build image
docker images                     # List images
docker run -d -p 8080:80 myapp    # Run container
docker ps                         # List running containers
docker stop <id>                  # Stop container
docker rm <id>                    # Remove container
```

→ **Real World Use Cases :**

- Microservices deployment
- CI/CD pipelines (Jenkins, GitHub Actions)
- Running databases (MySQL, Postgres)
- Local development environments
- Kubernetes (container orchestration)
- Cloud deployments (AWS, GCP, Azure)

→ **Production Ready Tips :**

- ✔ Use official/base images (e.g. node:18, openjdk:17)
- ✔ Keep images small (use multi-stage builds)
- ✔ Use .dockerignore to avoid unnecessary files
- ✔ Don't store secrets in images (use secrets manager)
- ✔ Scan images for vulnerabilities (Trivy / Docker Scout)
- ✔ Use tags (latest, v1.0.0) and push to a registry (Docker Hub / ECR)

Pro Tip :

"Docker makes your application environment consistent, portable and production ready."

Learn ♥ Build ♥ Grow Together ! Follow @the_devops_journey_with_sanket

Topic :- # Docker Architecture

Date :- 15 / 08 / 2025

"Understanding how Docker works behind the scenes"

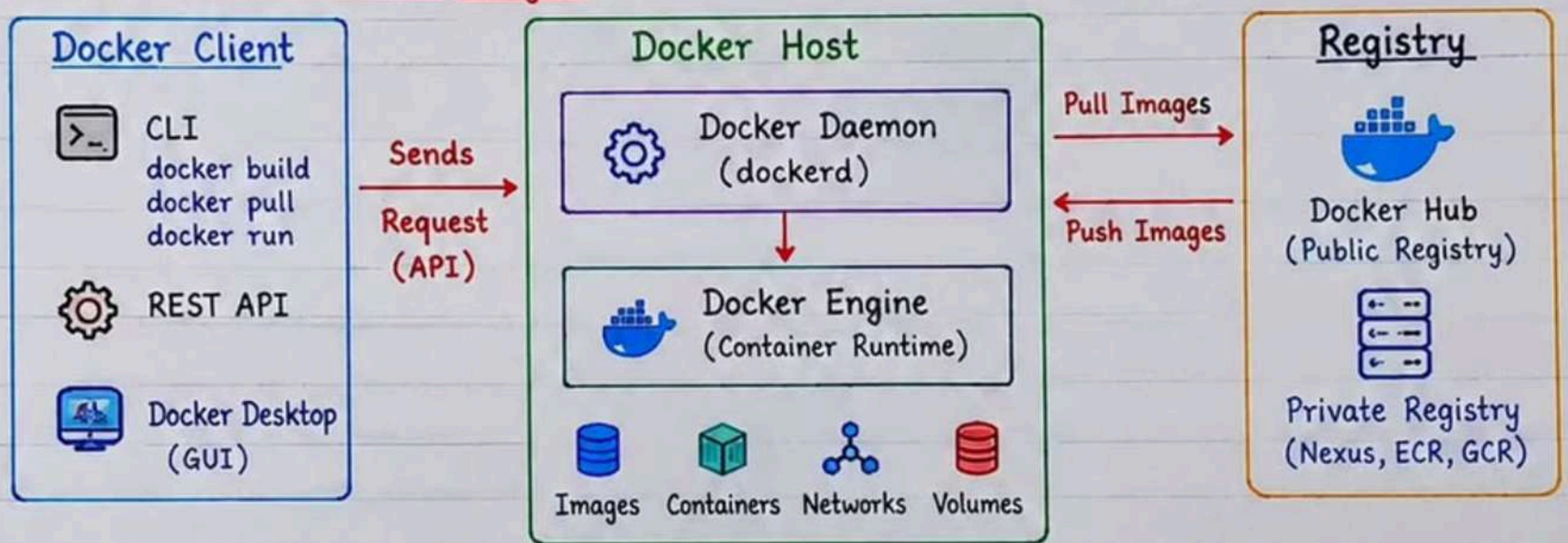
① What is Docker Architecture?

→ Docker uses a client-server architecture. The Docker client sends commands to the Docker daemon, which manages containers, images, networks and volumes on the Docker host. Images are pulled from / pushed to a container registry (e.g., Docker Hub).

Main Components:

- ☑ Docker Client (CLI / API)
- ☑ Docker Daemon (dockerd)
- ☑ Docker Host (Machine running Docker)
- ☑ Docker Engine (Core runtime)
- ☑ Registry (Docker Hub / Private)
- ☑ Images (Read-only templates)
- ☑ Containers (Running instances)

② Docker Architecture Diagram



③ Request Flow (Step by Step)

1. User runs a command (e.g. docker run nginx) on Docker Client.
2. Client sends the request to Docker Daemon (via REST API).
3. Daemon checks if the image exists locally.
4. If not, it pulls the image from the Registry (Docker Hub / Private).
5. Daemon uses Docker Engine to create and run the container.
6. Container runs on Docker Host using OS resources (namespaces, cgroups).
7. Output / logs are sent back to the client.

Key Points:

→ Client & Daemon communicate via REST API (Unix socket / TCP).
→ Docker Engine uses containerd + runc under the hood.
→ Images are read-only, containers are writable.
→ Registry stores & distributes Docker images.

④ Real World Analogy

→ Client	→ You (User sending order)
→ Daemon	→ Waiter (Manages request)
→ Engine	→ Kitchen (Runs the cooking process)
→ Registry	→ Supermarket (Stores ingredients / images)
→ Container	→ Food served (Running application)

⑤ Why this Architecture?

- ☑ Separation of concerns
- ☑ Secure & isolated environment
- ☑ Easy image distribution
- ☑ Works on any OS / Cloud
- ☑ Scalable & production ready

Build · Ship · Scale with Docker !

Follow for more DevOps notes ❤

@the_devops_journey_with_sanket

Learn Today...
Build Tomorrow...

@the_devops_journey_with_sanket

Docker Installation & Basics

Let's install Docker and run our first container ☺

① What is Docker ?

→ Docker is an open-source platform to build, ship and run applications in containers.
→ Containers are lightweight, portable and isolated environments for your app.
→ Docker helps in consistent environment from development to production.

Why Use Docker ?

✓ Lightweight
✓ Portable
✓ Consistent
✓ Faster Deployment
✓ Works on any OS

② Install Docker

On Ubuntu (Linux)

```
$ sudo apt update
$ sudo apt install -y docker.io
$ sudo systemctl start docker
$ sudo systemctl enable docker
$ sudo usermod -aG docker $USER
```

→ Log out and login again to apply group changes.

On Windows

1. Download Docker Desktop from → https://www.docker.com
2. Install and follow the setup
3. Start Docker Desktop
4. Verify using docker version

③ Check Docker Version

```
$ docker version
```

→ Shows client & server version.

④ Docker Info

```
$ docker info
```

→ Shows system info, storage, containers, etc.

⑤ Docker Help

```
$ docker --help
```

→ Shows all available docker commands.

⑥ Run Your First Container

```
$ docker run hello-world
```

→ Docker will pull the image from Docker Hub
→ Run the container
→ Show a success message ☺

Pro Tip :

✓ Use docker --help to explore more
✓ Use docker ps to list running containers
✓ Use docker images to list downloaded images

Small Steps... Big DevOps Journey !

#Docker #DevOps #Containers #Linux #Cloud #the_devops_journey_with_sanket ♥

Dockerfile — Fundamentals

A Dockerfile is a text file that contains instructions to build a Docker image. Docker reads this file and creates an image layer by layer.

🎯 Key Instructions

① **FROM** → Sets the base image for your container.
- First instruction in a Dockerfile
- Can use official or custom image

Example:
```
FROM node:20-alpine
```

② **RUN** → Executes a command in the image build process.
- Used to install packages / tools
- Creates a new layer

```
RUN apk add --no-cache
    curl git
```

③ **COPY** → Copies files/folders from local machine to the image.
- Use for app source code

```
COPY . /app
```

④ **ADD** → Similar to COPY, but has extra features
- Can download files from a URL
- Can auto-extract .tar files

```
ADD https://example.com/
    app.tar.gz /app
```

⑤ **WORKDIR** → Sets the working directory inside container.
- Creates the directory if not exists
- All subsequent commands run here

```
WORKDIR /app
```

⑥ **ENV** → Sets environment variables.
- Available during build & runtime

```
ENV NODE_ENV=production
ENV PORT=3000
```

⑦ **EXPOSE** → Documents the port the container will listen on.
- Does not publish the port (use -p at runtime)
- For documentation only

```
EXPOSE 3000
```

⑧ **CMD** → Default command to run when container starts.
- Can be overridden at runtime
- Use exec form (preferred)

```
CMD ["node", "server.js"]
```

⑨ **ENTRYPOINT** → Sets the main executable for the container.
- Not easily overridden
- Use for fixed entry scripts

```
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["node", "server.js"]
```

✅ Best Practices

- ✓ Use lightweight base images (alpine)
- ✓ Combine RUN commands to reduce layers
- ✓ Use .dockerignore file
- ✓ Prefer COPY over ADD (unless needed)
- ✓ Use multi-stage builds for smaller images
- ✓ Don't hardcode secrets in Dockerfile

Sample Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "server.js"]
```

⭐ Why Use Dockerfile?

- ✓ Reproducible builds
- ✓ Version controlled
- ✓ Portable across environments
- ✓ Consistent setup
- ✓ Easy to scale & deploy

Build Once, Run Anywhere ! 🚀

Date: 15-08-2025

@the_devops_journey_with_sanket

Dockerfile — Production Best Practices

Layer optimization, .dockerignore, non-root user, small images, security.

① Use a Small & Official Base Image

- ✓ Use official images (Docker Hub)
- ✓ Prefer slim/alpine versions
- ✓ Avoid latest tag (use specific version)

```
FROM eclipse-temurin:17-jre-alpine
```

↳ Small, secure & stable

② Optimize Layers (Better Caching)

- ✓ Copy files in right order
- ✓ Leverage layer caching
- ✓ Combine RUN commands
- ✓ Avoid unnecessary layers

```
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src/ ./src
RUN mvn package -DskipTests
```

③ Use .dockerignore

- ✓ Exclude unnecessary files
- ✓ Reduce build context size
- ✓ Build faster & smaller images

.dockerignore (example)

```
target/
.git/
node_modules/
*.log
.env
Dockerfile
```

④ Run as Non-Root User

- ✓ Never run as root
- ✓ Create a dedicated user
- ✓ Improve container security

```
RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup

USER appuser
WORKDIR /app
```

⑤ Keep Images Small

- ✓ Use multi-stage builds
- ✓ Install only required deps
- ✓ Remove cache/temp files
- ✓ Use alpine/slim images

```
# Stage 1 - Build
FROM maven:3.9-eclipse-temurin-17
AS  builder
RUN mvn package -DskipTests

# Stage 2 - Runtime
FROM eclipse-temurin:17-jre-alpine
COPY --from=builder /app/target/app.jar
     /app/app.jar
```

⑥ Security Best Practices

- ✓ Use minimal base image
- ✓ Run as non-root user
- ✓ Scan image for vulnerabilities
- ✓ Don't store secrets in image
- ✓ Use read-only filesystem (if possible)
- ✓ Keep dependencies updated

Scan Image:

```
docker scan myapp:latest
  or
trivy image myapp:latest
```

⭐ Production Ready Dockerfile (Example)

```
# Use small official base image
FROM eclipse-temurin:17-jre-alpine

# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Set work directory
WORKDIR /app

# Copy only required files
COPY target/app.jar  app.jar

# Change ownership
RUN chown -R appuser:appgroup /app

# Switch to non-root user
USER appuser

# Run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Quick Tips

- → Use specific version tags (e.g: 17.0.9)
- → Keep Dockerfile simple & clean
- → Use multi-stage builds
- → Remove cache (apk, apt) after install
- → Test image locally before push
- → Integrate image scanning in CI/CD

Result: Smaller ✓ Faster ✓ Safer ✓
Production Ready Image ✓

Follow for more DevOps Notes ! ♥

@the_devops_journey_with_sanket

Build Smart 🚀 Ship Secure 🚀 Go Production 🚀

Date : 15 Aug 2025

@the_devops_journey_with_sanket

Docker Images

Build Once → Run Anywhere

1) What is a Docker Image ?

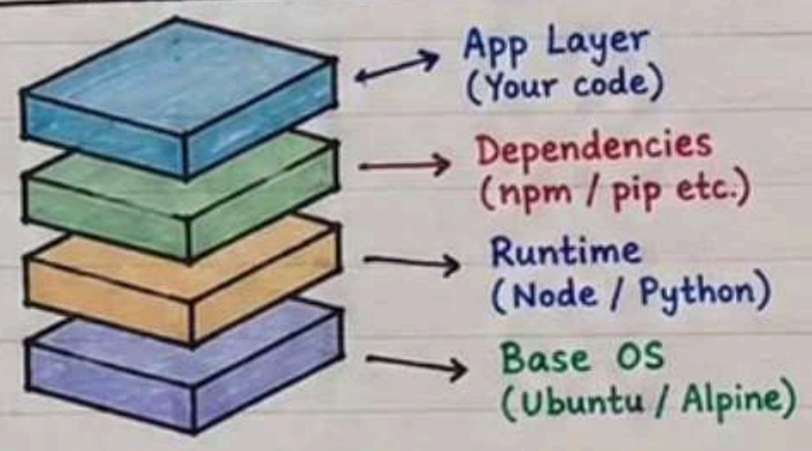
→ A read-only template used to create containers.
→ Contains application code, dependencies, runtime, configs etc.
→ Immutable (does not change).
→ Built in layers (Union File System).
→ Defined using a Dockerfile.

Think of it like:
An image is like a "blueprint" of your app. Container is a running instance of that blueprint.

2) Image Layers

→ Images are made up of multiple layers.
→ Each instruction in a Dockerfile creates a layer.
→ Layers are cached, so builds are faster.
→ Layers are read-only and shared across images (saves disk space).

Image Layers (Example)



3) Image ID

→ Unique identifier for an image.
→ Example: sha256:1a2b3c4d5e6f...
→ First 12 characters are usually shown.
→ Same image (content) will have same ID.

Example

```
IMAGE ID (Full) : sha256:1a2b3c4d5e6f7890...
IMAGE ID (Short): 1a2b3c4d5e6f
```

4) Tags

→ Human-friendly name for an image.
→ Format: <name>:<tag>
→ Default tag is latest (if not specified).
→ Helps manage different versions.

Example

```
nginx:latest   → latest version
nginx:1.25     → specific version
myapp:v1       → custom tag
```

5) Search Images

Search image on Docker Hub

```
docker search nginx
```

→ Shows available images
→ Use --limit to restrict results

```
docker search nginx --limit 5
```

6) Pull Images

Download image from registry

```
docker pull nginx:latest
```

→ Downloads image to local machine
→ Specify tag for exact version

7) List Images

Show all local images

```
docker images
```

→ Shows REPOSITORY, TAG, IMAGE ID, SIZE
→ Use -a to show all

```
docker images -a
```

8) Remove Images

Delete an image from local machine

```
docker rmi nginx:latest
```

→ Use image name or image ID
→ Use -f to force remove

```
docker rmi -f 1a2b3c4d5e6f
```

Pro Tips

✓ Use specific tags (avoid latest in prod)
✓ Keep images small (use alpine)
✓ Clean up unused images regularly
  `docker image prune -a`
✓ Use multi-stage builds for smaller images
✓ Scan images for vulnerabilities (Trivy / Docker Scout)

Real World Use Case

1. Search for base image
2. Pull required version
3. Build your custom image
4. Tag it with version
5. Push to Docker Hub / ECR
6. Use in CI/CD pipeline
7. Run as container in Dev / Prod

Keep Learning Keep Growing !

@the_devops_journey_with_sanket

Date : 15-Aug-2025

@the_devops_journey_with_sanket

Docker Build & Image Optimization

Build faster • Smaller images • Production Ready - --

① Docker Build (Basic)

Build an image from Dockerfile

```
docker build -t myapp:1.0 .
```

→ -t : tag name
→ . : build context (current dir)
→ Reads instructions from Dockerfile
→ Creates a new image

② Image Tags

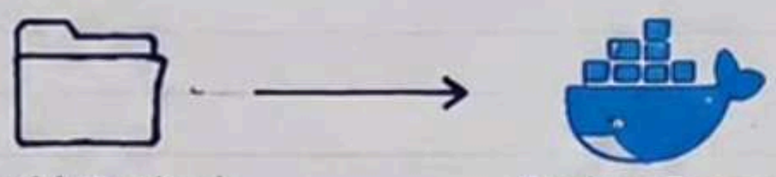
Tags help in versioning & managing images

```
docker build -t myapp:1.0 .
docker build -t myapp:latest .
docker tag myapp:1.0 myrepo/myapp:1.0
```

✓ Use meaningful tags (v1.0, v2.1)
✓ Avoid using only **latest** in production
✓ Use semantic versioning

③ Build Context

Files & folders sent to Docker daemon



→ Only files in build context are sent
→ Use .dockerignore to exclude files
→ Keep context small for faster builds

.dockerignore (example)

```
node_modules/
.git/
*.log
dist/
```

④ Build Cache

Docker caches layers to speed up builds

→ Reuses unchanged layers
→ Order instructions (less change first)
→ Use specific COPY instead of copying all

Tip:
✓ COPY package*.json first
✓ RUN npm install (cached if no change)
✓ Then copy rest of the code

⑤ Multi-Stage Builds

Use multiple stages to reduce final image size

```
# Stage 1 : Build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2 : Runtime
FROM nginx:alpine
COPY --from=builder /app/dist
     /usr/share/nginx/html
```

✓ Smaller final image
✓ No build tools in runtime
✓ More secure
✓ Production ready

⑥ BuildKit (Advanced)

Next-gen builder (faster + better caching)

```
# Enable BuildKit
export DOCKER_BUILDKIT=1
docker build -t myapp:1.0 .
```

Features :
✓ Faster builds
✓ Better caching
✓ Parallel execution
✓ Supports --secret, --mount

Use buildx for multi-platform builds

```
docker buildx build \
 --platform linux/amd64 .
```

Best Practices (Production Ready)

✓ Use .dockerignore
✓ Keep base image small (alpine/distroless)
✓ Use multi-stage builds
✓ Leverage build cache
✓ Use specific tags (avoid latest)
✓ Enable BuildKit / buildx
✓ Scan images for vulnerabilities

Smaller Image = Faster Deploy = Better Production !

@the_devops_journey_with_sanket

Keep Learning ... Keep Growing ...

Date : 12 Aug 2025

@the_devops_journey_with_sanket

Docker Containers

Container Lifecycle : Create → Start → Stop → Restart → Remove
Exec | Logs | Inspect

→ A container is a lightweight, portable and isolated runtime environment that runs an application with all its dependencies.

Create → Start → Run → Stop → Restart → Remove

① Create a container

```
docker create --name myapp nginx:latest
```

→ Creates a container (in stopped state)
→ Does not start automatically

② Start a container

```
docker start myapp
```

→ Starts an existing container

③ Run a container (create + start)

```
docker run -d --name myapp -p 8080:80 nginx:latest
```

→ Creates and starts a container in detached mode
→ -p → port mapping (host:container)

④ Stop a container

```
docker stop myapp
```

→ Gracefully stops the running container

⑤ Restart a container

```
docker restart myapp
```

→ Stops and starts the container again

⑥ Remove a container

```
docker rm myapp
```

→ Removes a stopped container
→ Use -f to force remove (running container)
docker rm -f myapp

⑦ Exec into a container

```
docker exec -it myapp /bin/bash
```

→ Access running container shell
→ Use /bin/sh if bash is not available

⑧ View container logs

```
docker logs myapp
```

→ Shows container logs
→ Use -f to follow logs in real time
docker logs -f myapp

⑨ Inspect container

```
docker inspect myapp
```

→ Shows detailed info in JSON format
→ Useful for debugging (IP, mounts, config, etc.)

Useful Commands

→ docker ps	→ List running containers
→ docker ps -a	→ List all containers
→ docker images	→ List local images
→ docker rm $(docker ps -aq)	→ Remove all stopped containers
→ docker system prune -f	→ Clean up unused resources

Pro Tips (Production Ready)

- ✓ Give meaningful container names
- ✓ Use -d for background run
- ✓ Check logs for debugging
- ✓ Use docker inspect for details
- ✓ Clean up unused containers & images
- ✓ Use docker compose for multi-container apps

Build • Automate • Scale • Repeat

Follow @the_devops_journey_with_sanket

Keep Learning ... Link in Bio !

Learn Today... Build Tomorrow...

Docker Volumes & Storage

Keep Your Data Safe, Persistent and Portable

1 Why Storage Matters ?

- Containers are ephemeral (data is lost when container stops).
- Use Volumes / Bind Mounts / tmpfs for persistent and reliable storage.
- Separate application code from data.
- Easy backup, restore and sharing across containers.

2 Types of Docker Storage

Volumes (Managed by Docker)
- Best for persistent data
- Stored in Docker's storage area

Bind Mounts (Host Path)
- Map host directory to container
- Good for dev / config files

tmpfs (In-Memory)
- Stored in RAM (no disk write)
- Fast & temporary (non-persistent)

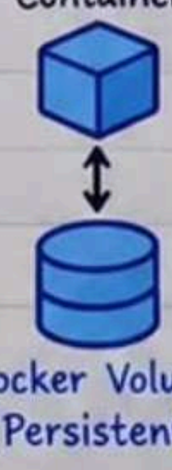
3 Docker Volumes (Recommended)

Create a volume:

```
docker volume create mydata
```

Run a container with volume:

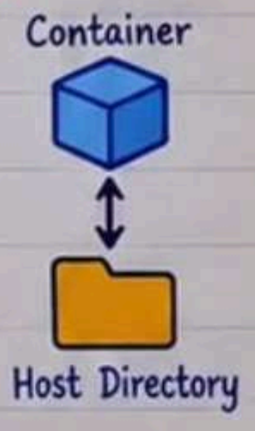
```
docker run -d \
  --name myapp \
  -v mydata:/app/data \
  nginx
```



4 Bind Mounts (Host Directory)

Run with bind mount:

```
docker run -d \
  --name myapp \
  -v /host/path:/app/data \
  nginx
```

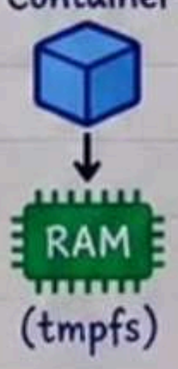


- ✓ Changes on host ↔ reflected in container
- ✓ Useful for development & config files
- ✓ You control the exact host path

5 tmpfs (In-Memory Storage)

Run with tmpfs:

```
docker run -d \
  --name myapp \
  --tmpfs /app/cache \
  nginx
```



- ✓ Stored in memory (no disk)
- ✓ Fast performance
- ✓ Data lost when container stops
- ✓ Use for cache / temp files / sensitive data

6 Backup & Restore Volumes

Backup a volume:

```
docker run --rm -v mydata:/data \
 -v $(pwd):/backup \
 alpine tar czf /backup/mydata_backup.tar.gz -C /data .
```

Restore a volume:

```
docker run --rm -v mydata:/data \
 -v $(pwd):/backup \
 alpine tar xzf /backup/mydata_backup.tar.gz -C /data
```

7 Best Practices

- ✓ Use named volumes for persistent data
- ✓ Keep app code & data separate
- ✓ Regularly backup important volumes
- ✓ Use tmpfs for sensitive / temporary data
- ✓ Set correct permissions (UID/GID)
- ✓ Use volume labels for easy management

Pro Tips:

- ✓ List volumes: `docker volume ls`
- ✓ Inspect volume: `docker volume inspect <name>`
- ✓ Remove volume: `docker volume rm <name>`
- ✓ Check storage usage: `docker system df -v`

Build Automate Scale Grow

@the_devops_journey_with_sanket

Learn Today... Build Tomorrow...

Date : 23 / 08 / 2025

Docker Registry

Store, Manage and Distribute Docker Images

1 What is Docker Registry ?

- A Docker Registry is a storage service that holds Docker images.
- Helps in sharing images across teams, environments and deployments.
- Docker Hub is a public registry.
- We can also run a private registry (on-prem / internal).

2 Types of Registry

Docker Hub (Public)

- Official public registry
- Free for public images
- docker.io (default)

Private Registry

- Self-hosted (e.g. using registry:2)
- Harbor, Nexus, GitLab Registry
- Used for internal / company images

3 Login to Registry

- **Docker Hub**

```
docker login
```

Enter your Docker Hub username & password

- **Private Registry**

```
docker login <registry-url>
```

Example:
docker login myregistry.company.com

4 Tag an Image

```
docker tag <image-name>:<old-tag>
<registry-url>/<image-name>:<new-tag>
```

Example:

```
docker tag myapp:1.0 \
   docker.io/yourusername/myapp:1.0
```

5 Push Image to Registry

```
docker push <registry-url>/<image-name>:<tag>
```

Example:
docker push docker.io/yourusername/myapp:1.0

6 Pull Image from Registry

```
docker pull <registry-url>/<image-name>:<tag>
```

Example:
docker pull docker.io/yourusername/myapp:1.0

7 Image Versioning (Best Practices)

- ✔ Use meaningful tags (e.g. v1.0.0, v1.1.0)
- ✔ Use latest only for development
- ✔ Use semantic versioning (MAJOR.MINOR.PATCH)
- ✔ Tag with commit hash for traceability
- ✔ Automate tagging in CI/CD pipeline

Examples:

- myapp:1.0.0
- myapp:1.1.0
- myapp:latest (not recommended for prod)
- myapp:abc123 (commit hash)

8 Run a Private Registry (Simple)

① Run registry container

```
docker run -d -p 5000:5000 --name registry \
   registry:2
```

② Tag and push

```
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
```

③ Pull from private registry

```
docker pull localhost:5000/myapp:1.0
```

Production Tips:

- ✔ Use private registry for sensitive images
- ✔ Enable authentication (TLS)
- ✔ Use image scanning (Trivy / Harbor)
- ✔ Set retention policy
- ✔ Use immutable tags
- ✔ Integrate with CI/CD (Jenkins / GitHub Actions)

Better Images... Smoother Deployments... Stronger Teams...

Keep Learning
Keep Building ♥

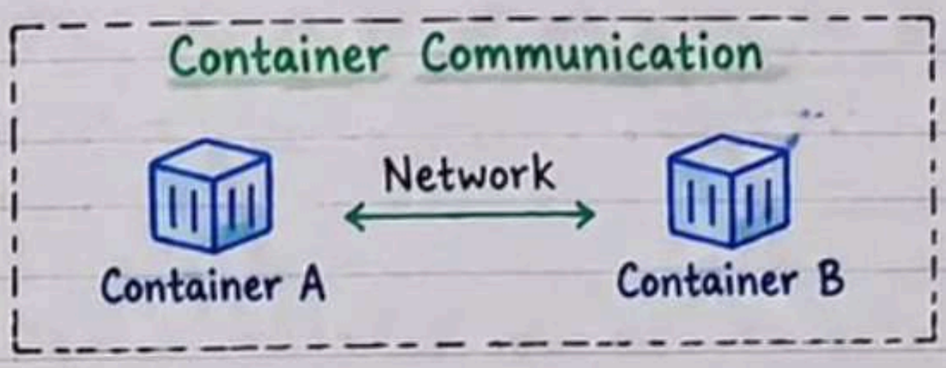
@the_devops_journey_with_sanket

Docker Networking

Connect Containers • Communicate • Build Scalable Apps

1 Why Docker Networking ?

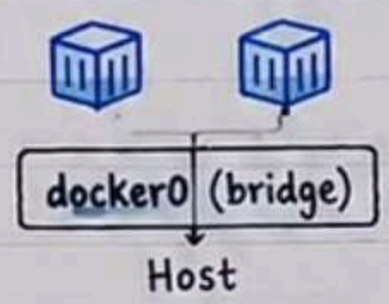
- ✓ Containers need to talk to each other
- ✓ Isolate apps for security
- ✓ Enable communication with external world
- ✓ Support scalable, microservices architectures



2 Types of Docker Networks

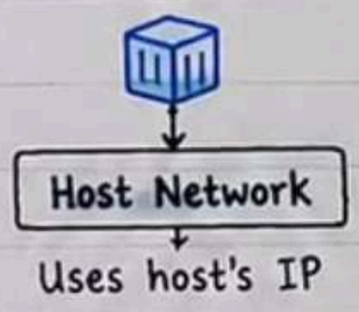
Bridge (Default)

- Default network
- Isolated on single host
- Uses NAT
- Containers can talk to each other



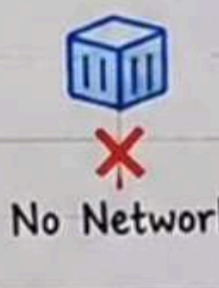
Host

- Shares host network namespace
- No NAT
- Faster performance
- Less isolation



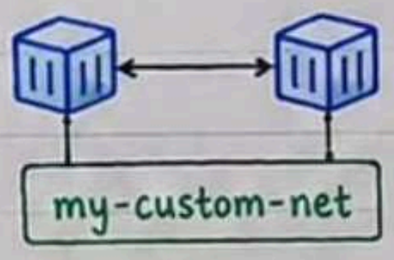
None

- No network
- Fully isolated
- No external or container communication
- Used for high security



Custom Network

- User-defined bridge
- Better control
- Built-in DNS
- Recommended for microservices



3 Create & Use Networks (Commands)

```
# Create a custom network
docker network create mynet   # creates a user-defined bridge

# Run containers in same network
docker run -d --name app1 --network mynet nginx
docker run -d --name app2 --network mynet nginx
                                          # both in mynet

# List networks
docker network ls

# Inspect network
docker network inspect mynet
```

4 Container-to-Container Communication

- ✓ Use the container name (DNS) to connect
- ✓ No need to use IP address
- ✓ Works out-of-the-box in custom networks

Example:

```
# inside app1 container
curl http://app2:80    # use container name
```

5 Docker DNS (Service Discovery)

- ✓ Docker provides built-in DNS for user-defined networks
- ✓ Containers can resolve each other by name
- ✓ Automatic name resolution, no need for /etc/hosts

Example DNS resolution:

```
ping app2    # works if both are in same network
```

6 Best Practices

- ✓ Use custom networks for production
- ✓ Avoid --network host in production (security risk)
- ✓ Use container names instead of IPs
- ✓ Segment networks for different services
- ✓ Use Docker Compose for multi-container apps

Pro Tip: Use custom networks + Docker Compose + service names for clean, scalable, and secure communication between containers!

Keep Learning
Keep Building
Together ❤

@the_devops_journey_with_sanket

Docker Compose

Run Multi-Container Apps with a Single Command

→ Docker Compose is a tool to define and run multi-container Docker applications using a single YAML file (compose.yaml). It simplifies managing services, networks, volumes, and environment variables for your application.

(1) Compose.yaml

- ✓ YAML file to define services
- ✓ Keeps infra as code
- ✓ Version 3.x is recommended
- ✓ Easy to share & maintain

(2) Basic Structure

```yaml
version: "3.9"
services:
  app:
    image: myapp:latest
    ports:
      - "8080:8080"
```

(3) Services

- ✓ Each container is a service
- ✓ Define image or build context
- ✓ Configure ports, env, volumes, etc.
- ✓ Multiple services can run together

(4) Networks

- ✓ Services communicate via network
- ✓ Automatically creates a default network
- ✓ You can also define custom networks

(5) Volumes

- ✓ Persist data outside containers
- ✓ Use named or bind volumes
- ✓ Useful for databases, configs, logs

(6) Environment Variables

- ✓ Use environment: or eny_file:
- ✓ Store config like DB URL, API keys
- ✓ Keeps secrets out of the image

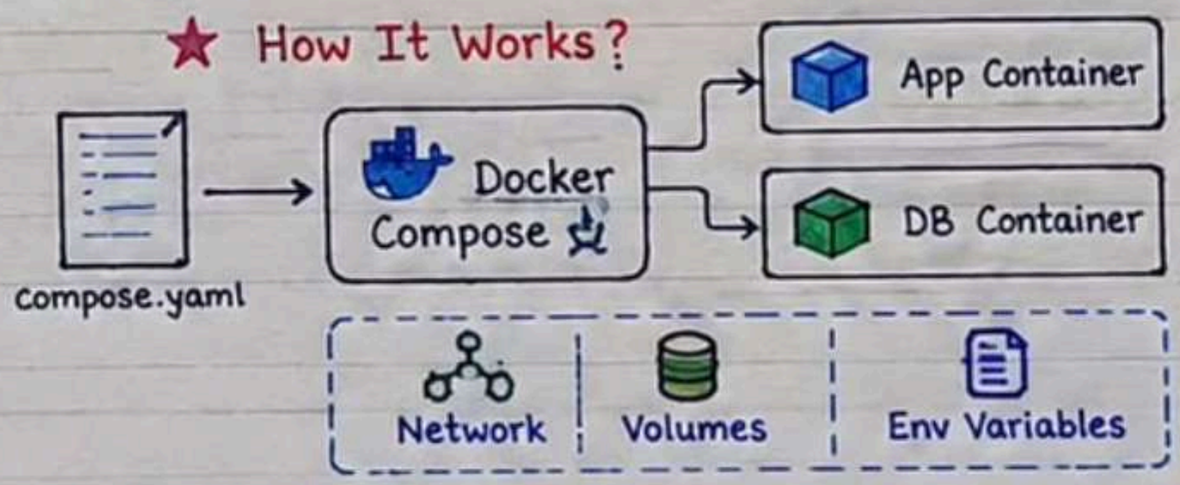
(7) Dependencies

- ✓ Use depends_on to control startup order
- ✓ Does not wait for service to be "ready" (only start order)
- ✓ For real readiness, use healthcheck

★ Complete Example (compose.yaml)

```yaml
version: "3.9"
services:
  app:
    build: ./app
    container_name: myapp
    ports:
      - "8080:8080"
    environment:
      - DB_URL=postgres://db:5432/appdb
      - APP_ENV=production
    depends_on:
      - db
    networks:
      - app-network
    volumes:
      - app-data:/app/data
  db:
    image: postgres:15
    container_name: postgres
    environment:
      - POSTGRES_DB=appdb
      - POSTGRES_USER=admin
      - POSTGRES_PASSWORD=admin123
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
networks:
  app-network:
    driver: bridge
volumes:
  app-data:
  db-data:
```

★ How It Works?



★ Useful Commands

```
→ docker compose up -d      # Start in background
→ docker compose ps         # Check running services
→ docker compose logs -f    # View logs
→ docker compose down       # Stop & remove
→ docker compose build      # Rebuild images
```

☆ Best Practices

- ✓ Use .env file for sensitive values
- ✓ Use named volumes for persistence
- ✓ Add healthcheck for critical services
- ✓ Use specific image versions (not latest)
- ✓ Keep compose.yaml in version control (Git)

Simplify • Automate • Ship Faster !

@the_devops_journey_with_sanket

Docker Compose Production Setup

App + DB architecture, healthchecks, secrets, restart policies, resource limits

1) Architecture (App + DB)



- ✓ Docker Compose
- ✓ Isolated Network
- ✓ Healthchecks
- ✓ Secrets
- ✓ Persistent Volume
- ✓ Secure Credentials
- ✓ Restart Policy
- ✓ Resource Limits

2) Project Structure

```
docker-compose.yml
.env (keep out of git)
secrets/
  ├→ db_password.txt
  └→ app_secret.txt
app/
  ├→ Dockerfile
  └→ app code ...
```

Use .env for non-sensitive values
Use Docker secrets for sensitive data (in production)

3) Healthchecks

- ✓ Ensures container is actually ready
- ✓ Compose waits for DB to be healthy
- ✓ Prevents app from failing on startup

Example (Postgres):

```yaml
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
  interval: 10s
  timeout: 5s
  retries: 5
```

4) Restart Policies

- ✓ Keeps containers running in case of failure
- ✓ Best for production stability

Common options:

- no – Do not restart
- on-failure – Restart only if it fails
- always – Always restart
- unless-stopped – Restart unless manually stopped (Recommended)

5) docker-compose.yml (Production Ready)

```yaml
version: '3.9'
services:
  app:
    build: ./app
    container_name: myapp
    ports:
      - "8080:8080"
    env_file:
      - .env
    secrets:
      - app_secret
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
    deploy:
      resources:
        limits:
          cpus: "0.50"
          memory: "512M"
        reservations:
          cpus: "0.25"
          memory: "256M"
  db:
    image: postgres:16-alpine
    container_name: mydb
    environment:
      POSTGRES_DB: myappdb
      POSTGRES_USER: myappuser
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes:
      - db_data:/var/lib/postgresql/data
    secrets:
      - db_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
secrets:
  db_password:
    file: ./secrets/db_password.txt
  app_secret:
    file: ./secrets/app_secret.txt
volumes:
  db_data:
```

6) Resource Limits (Why?)

- ✓ Prevents one container from using all system resources
- ✓ Improves stability & performance
- ✓ Helps in multi-service environments

7) Best Practices (Production)

- ✓ Use .env for config (not secrets)
- ✓ Use Docker secrets for sensitive data
- ✓ Add healthchecks for all services
- ✓ Set restart: unless-stopped
- ✓ Use resource limits
- ✓ Use named volumes for persistent data
- ✓ Keep images minimal (alpine/distroless)
- ✓ Use separate network (default in compose)
- ✓ Monitor logs (docker compose logs -f)

Final Tip:

Docker Compose is great for small to medium production setups. For large scale, consider orchestration tools like Kubernetes.

Build • Automate • Deploy • Grow →

@the_devops_journey_with_sanket

Docker Security

Build Secure Images • Run Safer Containers • Follow Least Privilege

Non-root containers, capabilities, read-only FS, image scanning, secrets

1) Use Non-Root Containers

→ Never run containers as root.
→ Use non-root user for better security.
→ Create a dedicated user in Dockerfile.

```
FROM node:20-alpine
RUN addgroup -S appgroup  &&
    adduser -S appuser -G appgroup
USER appuser
WORKDIR /app
COPY . .
CMD [ "node", "app.js" ]
```

→ Run as non-root user ✓

2) Limit Linux Capabilities

→ Drop all unnecessary capabilities.
→ Add only what is required.

```
docker run \
    --cap-drop ALL \
    --cap-add NET_BIND_SERVICE \
   myapp:latest
```

•• Use --cap-drop ALL for safest setup ✓

3) Use Read-Only File System

→ Mount container filesystem as read-only.
→ Allow write access only where needed.

```
docker run --read-only \
      --tmpfs /tmp \
      --tmpfs /var/run \
       myapp:latest
```

→ Prevents malicious writes ✓

4) Scan Images for Vulnerabilities

→ Scan images before using in production.
→ Use trusted tools (Trivy, Docker Scout).

```
# Using Trivy
trivy image myapp:latest

# Using Docker Scout
docker scout cves myapp:latest
```

✓ Find CVEs
✓ Fix issues
✓ Use minimal base images

5) Use Secrets (Don't Hardcode)

→ Use Docker secrets for sensitive data.
→ Never store passwords / API keys in image.

Create and use secret:

```
echo "mysecret" | docker secret create db_pass -
```

```
# docker-compose.yml
services:
    app:
        image: myapp
        secrets:
            - db_pass
secrets:
    db_pass:
        external: true
```

Access inside container:

/run/secrets/db_pass ✓

6) Follow Least Privilege

✓ Use non-root user
✓ Drop unnecessary capabilities
✓ Use read-only FS
✓ Allow only required network access
✓ Use minimal base images (alpine/distroless)
✓ Restrict with security policies (AppArmor/SELinux)
✓ Keep images updated regularly

7) Extra Security Tips

- Use official & trusted base images
- Keep Docker & host OS updated
- Don't expose unnecessary ports
- Enable logging & monitoring
- Use image signing (Docker Content Trust)

Key Takeaway:

Security is not a feature, it's a practice! Use least privilege, scan regularly, and keep secrets safe.

Build Secure • Deploy Confident • Keep Learning !

@the_devops_journey_with_sanket ♥

Date : 12 Aug 2025

@the_devops_journey_with_sanket

Docker Commands Cheat Sheet

Most-used Docker Commands | Practical Examples | Debugging Tips

Learn • Practice • Use in Real Projects

① Images (Build, Pull, List, Remove)

```
docker pull nginx              # Pull image from Docker Hub
docker images                  # List all local images
docker build -t myapp:1.0 .    # Build image
docker rmi myapp:1.0           # Remove image
docker image prune -f          # Remove unused images
```

② Containers (Lifecycle, Stop, Remove)

```
docker run -d --name web nginx  # Run container
docker ps                       # List running containers
docker ps -a                    # List all containers
docker stop web                 # Stop container
docker start web                # Start container
docker restart web              # Restart container
docker rm web                   # Remove container
docker rm -f web                # Force remove
```

③ Exec & Logs

```
docker exec -it web /bin/bash   # Open shell
docker exec web ls -l /app      # Run command
docker logs web                 # View logs
docker logs -f web              # Follow logs (live)
docker logs --tail 100 web      # Last 100 lines
```

④ Inspect & Info

```
docker inspect web              # Detailed info (JSON)
docker inspect nginx            # Image details
docker port web                 # Show port mapping
docker top web                  # Running processes
docker stats                    # Live resource usage
```

⑤ Volumes (Data Persist)

```
docker volume create mydata          # Create volume
docker volume ls                     # List volumes
docker run -d -v mydata:/app/data nginx
docker volume rm mydata              # Remove volume
docker volume prune -f               # Remove unused
```

⑥ Networks

```
docker network ls                    # List networks
docker network create mynet          # Create network
docker run -d --name web --network mynet nginx
docker network inspect mynet         # Network details
docker network rm mynet              # Remove network
```

⑦ Debugging (Troubleshoot)

```
docker ps -a                         # Check if container exited
docker logs <container>              # Check logs
docker inspect <container>           # Find error details
docker exec -it <container> sh       # Try /bin/sh if bash not found
docker stats                         # Check CPU / Memory
docker events                        # Real-time events
docker system df                     # Disk usage
```

⑧ Cleanup (Keep it clean)

```
docker container prune -f            # Remove stopped containers
docker image prune -a                # Remove unused images
docker volume prune -f               # Remove unused volumes
docker network prune -f              # Remove unused networks
docker system prune -a               # Full cleanup
```

Pro Tips

- ✓ Use meaningful names → docker run --name myapp ...
- ✓ Use tags → myapp:1.0, myapp:latest
- ✓ Check logs first when container fails
- ✓ Use docker inspect for deep debugging
- ✓ Clean unused resources regularly

Common Issues & Fixes

- → Container exits immediately → Check logs (docker logs)
- → Port already in use → Change host port (e.g., -p 8081:80)
- → Permission denied → Add sudo or check volume perms
- → No space left → Run docker system prune -a
- → App not reachable → Check network & port mapping

Build • Run • Debug • Ship

Follow @the_devops_journey_with_sanket

Keep Learning ... Link in Bio !

Docker Environment & Secrets

ENV, .env, Docker secrets, configuration management, sensitive data handling

1) What are Environment Variables ?

→ Key-value pairs used to pass configuration to containers.
→ Helps separate config from code.
→ Useful for different environments (dev / staging / prod).

2) Using ENV in Dockerfile

```
FROM node:18
ENV APP_NAME=myapp
ENV PORT=3000
ENV LOG_LEVEL=info
CMD [ "node", "app.js" ]
```

3) Using .env file with Docker

→ Create .env file (in project root)

```
DB_HOST=postgres
DB_USER=appuser
DB_PASSWORD=strongpassword
APP_ENV=production
```

→ Run container with .env file

```
docker run --env-file .env myapp
```

4) Using Environment Variables with Docker Compose

```
version: '3.8'
services:
  app:
    image: myapp:latest
    env_file:
      - .env
    environment:
      - PORT=3000
      - LOG_LEVEL=info
```

5) Docker Secrets (Swarm Mode)

→ Used for sensitive data (passwords, API keys).
→ Stored securely in Docker (encrypted).
→ Mounted as files inside container.

```
# Create a secret
echo "mysecretpassword" | docker secret create db_password -
```

```
# Use in service (docker-compose.yml)
secrets:
  - db_password
services:
  app:
    image: myapp
    secrets:
      - db_password
```

→ Inside container:
Secret available at:
/run/secrets/db_password

6) Best Practices

✓ Never hardcode sensitive data
✓ Use .env file for non-sensitive config
✓ Use Docker secrets for passwords / keys
✓ Use external secret managers (AWS Secrets Manager, HashiCorp Vault)
✓ Use different configs for dev / staging / prod
✓ Restrict access with proper permissions
✓ Rotate secrets regularly

7) Configuration Management

→ Use .env for local development
→ Use Docker secrets / Vault for production
→ Use config files / mounted volumes for complex config
→ Keep config outside the image (12 factor app)

Key Takeaway:

Keep your code clean, config separate, and secrets secure !
Use the right tool for the right environment.

Build Secure • Deploy Confident • Keep Learning !

@the_devops_journey_with_sanket ♥

Docker Resource Management

Control & optimize CPU, Memory, Disk usage of containers for better performance, stability and cost efficiency.

① CPU Limits & Reservations

- --cpus : Limit CPU usage (in cores)
- --cpu-shares : Relative weight (reservation)
- --cpuset-cpus : Bind to specific CPU cores

```
docker run -d \
  --name myapp \
  --cpus="1.5" \          # limit to 1.5 cores
  --cpu-shares=512 \      # reservation (relative)
  --cpuset-cpus="0,1" \   # use only core 0 & 1
  nginx:latest
```

Tip:

- Use limits to avoid noisy neighbor issue.
- Use reservations for fair scheduling.
- In Swarm/K8s, use deploy.resources for more control.

② Memory Limits & Reservations

- --memory : Hard limit (e.g. 512m, 1g)
- --memory-reservation : Soft limit (reservation)

```
docker run -d \
  --name myapp \
  --memory="512m" \                  # hard limit
  --memory-reservation="256m" \ # soft limit
  nginx:latest
```

③ OOM (Out of Memory)

- If container exceeds memory limit → Linux kernel kills the container (OOM Kill).
- Check exit code: 137 (OOM)
- Prevent by setting proper memory limits and monitoring usage.

```
docker inspect <container_id> |
grep -i oom
```

④ Check Resource Usage (Stats)

- Real-time CPU / Memory / Network / Block I/O

```
docker stats
docker stats <container_id>
```

CONTAINER	CPU %	MEM USAGE/LIMIT	NET I/O	BLOCK I/O
myapp	0.52%	120MiB / 512MiB	1.2MB / 1.1MB	3.5MB / 1.1MB
nginx	0.21%	45MiB / 256MiB	0.8MB / 0.6MB	1.2MB / 0.4MB

⑤ Disk Usage

- Check space used by images, containers, volumes
- Helps find what is consuming disk

```
docker system df
docker system df -v
docker ps -s          # size per container
```

⑥ Container Cleanup

- Remove stopped containers

```
docker container prune -f
```

- Remove unused images

```
docker image prune -a -f
```

- Full cleanup (containers, images, networks, build cache)

```
docker system prune -a --volumes -f
```

Best Practices

- ✓ Always set CPU & memory limits in production.
- ✓ Monitor with docker stats / Prometheus + Grafana.
- ✓ Use healthchecks to detect stuck containers.
- ✓ Clean up unused resources regularly.
- ✓ Use smaller base images to save disk space.

Remember:

- Limits = Maximum allowed
- Reservations = Minimum guaranteed
- OOM Kill = Exit code 137
- Clean up = Better disk space & performance

Build • Monitor • Optimize • Grow

@the_devops_journey_with_sanket

Docker Logging & Troubleshooting

Find issues faster · Keep containers healthy · Run production ready ☑

① Docker Logs (View Container Logs)

- View real time logs
  ```
  docker logs <container_name>
  ```
- Follow logs (live)
  ```
  docker logs -f <container_name>
  ```
- Show last 100 lines
  ```
  docker logs --tail 100 <container_name>
  ```
- Since specific time
  ```
  docker logs --since 1h <container_name>
  ```

② Logging Drivers

Docker supports multiple logging drivers:

- → json-file → Default driver (stores logs in files)
- → local → Better performance, less disk usage
- → syslog → Send logs to syslog server
- → journald → Use systemd journal (Linux)
- → awslogs → Send logs to AWS CloudWatch
- → fluentd → Send logs to Fluentd
- → gelf → Send logs to Graylog

Set logging driver while running container:

```
docker run --log-driver=awslogs \
  --log-opt awslogs-region=ap-south-1 \
  --log-opt awslogs-group=my-app  nginx
```

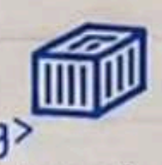
③ Container Debugging



- → Check container status
  `docker ps -a`
- → Inspect container (details)
  `docker inspect <container_name>`
- → Enter container shell
  `docker exec -it <container_name> sh`
- → Check resource usage
  `docker stats`
- → Check processes inside
  `docker exec -it <container_name> ps aux`
- → Check container logs + errors
  `docker logs -f <container_name>`

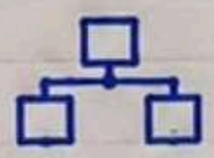
Inspect → Check → Fix → Repeat

④ Image Related Issues



- → Image not found → Check image name & tag
  `docker pull <image:tag>`
- → Wrong image version → Use specific tag (avoid latest)
  `docker pull nginx:1.25`
- → Large image size → Use multi-stage build & .dockerignore
- → Corrupted image → Remove & pull again
  `docker rmi <image>`
  `docker pull <image>`
- → Build failure → Check Dockerfile, context & logs
  `docker build --no-cache -t my-app .`

⑤ Network Related Issues



- → Container cannot reach internet
  - Check network mode (bridge/host)
  - Verify DNS → `docker exec <c> nslookup google.com`
  - Check proxy / firewall rules
- → Container to container communication
  - Use custom network
    `docker network create my-network`
  - Attach containers to same network
- → Port not accessible
  - Check port mapping → `docker ps`
  - Verify security group / firewall
  - Check if app is listening on 0.0.0.0

⑥ Disk / Storage Issues



- → Check disk usage
  `docker system df`
- → Clean up unused data
  ```
  docker system prune -a
  docker volume prune
  ```
- → Logs taking too much space
  - Use local logging driver
  - Set log rotation
    ```
    --log-opt max-size=10m
    --log-opt max-file=3
    ```

Production Tips ☑

- ✓ Use structured logging (JSON)
- ✓ Ship logs to CloudWatch / ELK / Loki
- ✓ Set log rotation to avoid disk full
- ✓ Monitor container health & resource usage
- ✓ Use specific image tags (avoid latest)
- ✓ Keep images small (multi-stage build)

★ Common Commands (Quick Ref)

`docker ps -a`	→ List containers	`docker stats`	→ Resource usage
`docker logs -f <c>`	→ Follow logs	`docker exec -it <c> sh`	→ Access shell
`docker inspect <c>`	→ Full details	`docker system df`	→ Disk usage

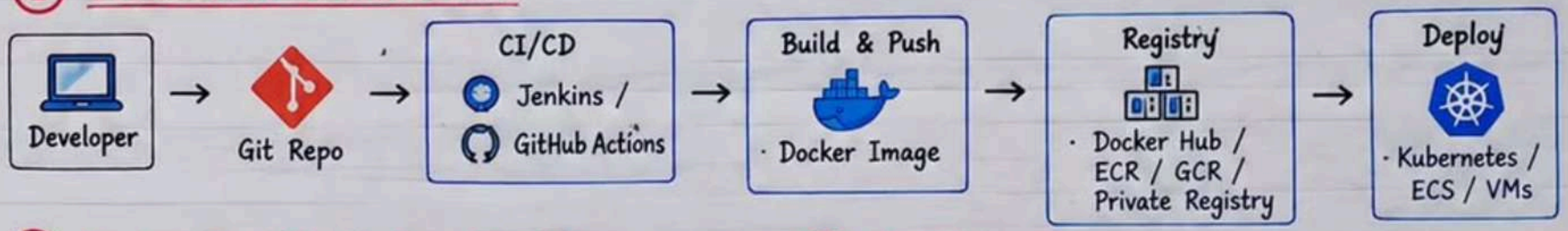
Debug Smart → Save Time → Keep Learning

Follow @the_devops_journey_with_sanket
For more DevOps Notes & Tips! ♥

Production Docker + Interview Cheat Sheet

Production architecture, best practices, common failures, Docker interview questions

① Production Architecture



② Dockerfile (Production Ready)

```
FROM eclipse-temurin:17-jre-alpine   # small & secure
WORKDIR /app    .
COPY target/app:jar app.jar

RUN addgroup -S app && adduser -S app -G app
USER app    # non-root user
EXROSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

③ Best Practices

- ✔ Use official & minimal base images (alpine)
- ✔ Keep images small (use .dockerignore)
- ✔ Do not run containers as root
- ✔ Use multi-stage builds
- ✔ Scan images for vulnerabilities
- ✔ Use specific image tags (avoid latest)
- ✔ Store secrets using Secrets Manager / ENV
- ✔ Use health checks
- ✔ Set resource limits (CPU / Memory)
- ✔ Use read-only filesystem (when possible)
- ✔ Log to stdout / stderr
- ✔ Automate builds with CI/CD

④ Useful Docker Commands

```
$ docker build -t myapp:1.0 .      # Build image
$ docker run -d --name app -p 8080:8080 myapp:1.0
$ docker ps                        # List containers
$ docker logs -f app               # View logs
$ docker exec -it app sh           # Access container
$ docker stats                     # Resource usage
$ docker system prune -a           # Clean up
```

⑤ Common Failures & Fixes

Problem		Fix
Container exits immediately	→	Check logs (docker logs), entrypoint / CMD
Image size too large	→	Use alpine, multi-stage build
Port already in use	→	Change host port (e.g. 8081:8080)
Permission denied	→	Use non-root user / correct file perms
Out of memory	→	Set memory limits / optimize app
Can't pull image	→	Check registry auth / network
App not reachable	→	Check EXPOSE, port mapping, firewall

⑥ Docker in Production (Extras)

- Use Container Orchestration → Kubernetes / ECS
- Use Private Registry → ECR / Harbor / Nexus
- Use Rolling Updates → Zero downtime
- Add Health Checks → Monitor container health
- Use Centralized Logging → ELK / CloudWatch / Loki
- Use Image Scanning → Trivy / SonarQube / Snyk

⑦ Top Docker Interview Questions

1. What is a Docker container and how is it different from a VM?
2. What happens when you run 'docker run'?
3. What is the difference between COPY and ADD?
4. What are Docker layers? How do they help?
5. How do you reduce the size of a Docker image?
6. What is a multi-stage build?
7. How do you persist data in Docker?
8. What is Docker Compose? When do you use it?
9. How do you handle secrets in Docker?
10. What are common security best practices in Docker?

⑧ .dockerignore (Example)

```
node_modules/
target/
.git/
*.log
Dockerfile
.env
```

Smaller
Safer
Faster
Production Ready!

Key Takeaway:

" Use Docker the right way —
Build small, secure, and scalable
images for production success! "

@the_devops_journey_with_sanket

Follow | Like | Keep Learning!